

Programming Project 1: Sequence Alignment

CS 181, Fall 2026

Out: Oct. 3

Due: Oct. 16, 11:59 PM

1 Task

You must implement the following three alignment algorithms you've learned in class:

1. Global Alignment
2. Local Alignment
3. Global Alignment with Affine Gap Penalties

Each algorithm should take in two strings u and v , a scoring matrix, and a gap penalty (or penalties in the case of affine gap). The scoring matrix will be a table that gives scores for the match or mismatch of two letters. More details on the precise form of the inputs will be given in the following section.

After implementing these algorithms, you will use them to perform alignments using real sequence data.

Note: There is an application section described below. Make sure to start early so you are not crunched for time closer to the deadline!

2 Program Specifications

2.1 Setup

To grab the support code, run `git pull` in your CS181 projects directory. This will fill in the stencil code in the `project1` directory.

2.2 Programming Language

- You may use any of the following programming languages to write your code for this class: Python 3, Java, R, and Julia. Our Gradescope autograder is configured to accept solutions written in these.
- Your solutions generally should not require the installation of any packages that do not come in the standard installations of your chosen programming language. However, if you are using Python, you will also have access to `numpy` and `pandas`.
- To facilitate anonymized and automated grading, each of your solutions must be accompanied by a shell script. Make sure each problem is able to output the correct result using the shell script provided. If you are using Python 3, your shell script must run your program using the command `"python3"` rather than `"python"`, as `"python"` will run your code with Python 2.

- Your shell scripts should print exactly what is shown in the examples given for each problem. If you print any extra text, you will fail our autograder and lose points. This means that if you are coding in R, you may need to print text using “cat()” instead of “print()”.
- Be sure to print any terminal output to stdout, which is the channel that the standard print functions write to in most programming languages. If you print to stderr, your solution will be interpreted as an error message and fail the autograder.

2.3 Alignment Characters

Use ‘-’ to represent gap characters. Your project should also be case-insensitive, so ‘a’ and ‘A’ should be equivalent.

2.4 Shell Scripts And Examples

Your handin should provide three shell scripts: `global.sh`, `local.sh`, and `affine.sh`. Each script should take three arguments: (1) a path (relative or absolute) to a file with the two sequences to align, (2) a path to a file with the scoring matrix, and (3) a gap penalty (or penalties, in the case of `affine.sh`). Your script should then print the first string aligned and then the second string aligned, followed by the alignment score, as an integer. Examples of the input files are shown below.

The sequences file just has two sequences on two separate lines. For example, here are the contents of `test_cases/01.txt`. Note that input files may or may not have newlines at the end of line 2.

```
AAAGAATTCA
AAATCA
```

The scoring matrix file has rows on separate lines and columns delimited by a single space. (The top left cell will be a simple ‘X’, to make parsing the matrix a little simpler.) An example scoring matrix can be found at `matrices/standard.m`. All test case matrices will follow the same format.

```
X C T A G
C 1 -1 -1 -1
T -1 1 -1 -1
A -1 -1 1 -1
G -1 -1 -1 1
```

Here are a few examples illustrating how we expect to be able to call your shell scripts and what we expect as output.

```
> sh global.sh test_cases/01.txt matrices/standard.m -1
AAAGAATTCA
AAA---T-CA
2
```

```
> sh local.sh test_cases/01.txt matrices/standard.m -1
AATTCA
AAT-CA
4
```

```
> sh affine.sh test_cases/01.txt matrices/standard.m -1 0
AAAGAATTCA
AAA----TCA
5
```

Note that `affine.sh` requires a gap opening penalty and a gap extension penalty (**in that order!**).

2.5 README

All projects must include a README file. Include the following information:

- A description of any known bugs.
- Anything you want the TAs to know about your project.

3 Application: ExploreInformatics Part 1

Please go to [this link](#) to access the application material. Please keep in mind that this section should take around 3 hours to complete. You will have to share your findings/results in a document titled **application.pdf**. You **must** include this in your submission for feedback.

4 Handin

- To hand in this assignment, upload your shell scripts along with all files needed to run your code on Gradescope. If you have any subdirectories or folders that you would like to preserve in your handin, you will need to compress your submission into a zip file and upload the zip file to Gradescope. However, to ensure that the autograder runs your handin properly, make sure that all shell scripts are present at the root of your handin; in other words, do not place your shell scripts inside any folders.
- When you hand in your solution, our autograder will immediately run on your submission. We have made the test cases provided in the project handout immediately visible to you so that you can ensure your solution runs properly with the autograder. If you fail the autograder and cannot determine why, notify a TA and we will help you to diagnose the problem. If your final submission is not compatible with our autograder, it will be very difficult for us to give you credit for this assignment.
- **Application Handin:** Make sure to submit the following: `align_reads.sh`, `student_chipseq.r`, `application.pdf`, and `abridged_counts.txt.summary`. To use the `zip_submission.sh` script, make sure to put `align_reads.sh` and `student_chipseq.r` in the `scripts` folder, `abridged_counts.txt.summary` in the `outputs` folder, and `application.pdf` in the `project1` folder. Your scripts for global, local, and affine gap alignment should be in the `alignment`

folder as well. You can then run `./zip_submission.sh 1` from the `cs181-user` folder to obtain your `project1_submission.zip` file.

5 Grading

We will grade your handin using pre-generated test cases, with a certain number of points allocated per test case. Your code should not throw exceptions for any valid inputs. Test edge cases extensively! We will be checking for common gotchas, including but not limited to the following:

1. Global alignment of two very short strings, e.g. 'C' and 'G', using various scoring schemes.
2. Local alignment of two very short strings, e.g. 'C' and 'G', using various scoring schemes.

You will be graded solely on the correctness of your implementation. You will not be graded on code performance, as long as it is within reasonable bounds. For this project, we will give a lenient limit of one minute for aligning two 500bp sequences. (If implemented wisely, these alignments should take less than half a second to perform.)

6 Tips

- We do not provide you with pseudocode for the algorithms in this handout. Instead, you should refer to the lecture notes/slides.
- If you are having trouble with the shell scripts or with parsing the input files, try temporarily hard-coding some test cases and scoring schemes and returning to parsing later or come to office hours. We want you to focus on the implementation of the algorithm, not the technical details of I/O.
- In the past, students have had more problems with backtracking than with filling out the dynamic programming table. Think carefully about what you need to do to backtrack. Again, feel free to come to office hours if you need help checking your logic.